

Blinkids

Guía de Migración y Cuentas Necesarias

Preparado para la entrega del proyecto

07 de August de 2026

Este documento tiene dos partes: primero, la lista de cuentas/servicios externos de los que depende la app y que el equipo que la reciba debe registrar o tomar control; segundo, una revisión del estado del proyecto de cara a la entrega, con los puntos que hay que resolver antes o justo después de migrarlo.

Resumen rápido

- 6 cuentas/servicios externos identificados (Supabase, GitHub, Google, OneSignal, Google Play, Apple Developer).
- 2 puntos críticos que deben resolverse antes de entregar: cambios locales sin subir a GitHub, y el build de Android firmado con la llave de debug.
- No se encontraron llaves secretas expuestas en el código de la app (la anon key de Supabase y el App ID de OneSignal están diseñados para ir embebidos en el cliente).

1. Cuentas que deben registrarse

Por cada servicio: para que se usa, el dato que ya existe en el proyecto (para identificarlo), y los pasos concretos para tomar control de el.

1. Supabase (backend, base de datos, autenticación)

CRITICO

Para que sirve en el proyecto:

Es todo el backend de la app: base de datos Postgres, autenticación de usuarios (padres e hijos), reglas de seguridad (RLS), funciones RPC que mueven monedas entre billeteras, y una función Edge que envía las notificaciones push.

Dato actual encontrado en el proyecto:

Proyecto:	mzwvazjofbdlsejxoqi
URL:	https://mzwvazjofbdlsejxoqi.supabase.co

Que tiene que hacer el equipo que recibe el proyecto:

- Crear una cuenta en supabase.com con el correo del nuevo equipo/organización.
- Pedirle al dueño actual que agregue esa cuenta como miembro del proyecto (Project Settings > Team), o que transfiera el proyecto completo a la nueva organización.
- Una vez con acceso, revisar Authentication > Providers > Google: ahí están guardadas las credenciales de Google OAuth (client ID/secret) - no están en el código de la app.
- Revisar Edge Functions > send-push > Secrets: debe tener configuradas ONESIGNAL_APP_ID y ONESIGNAL_REST_API_KEY (ver punto 4).
- Si se decide NO quedarse con este mismo proyecto de Supabase, hay un backup completo de esquema/funciones/políticas/datos en la carpeta supabase_backup/ del repositorio, mas la carpeta supabase/ con las migraciones aplicadas.

2. GitHub (repositorio de código)

CRITICO

Para que sirve en el proyecto:

Aquí vive todo el código fuente de la app (Flutter) y el historial de cambios.

Dato actual encontrado en el proyecto:

Repositorio:	github.com/merinogomez17-sudo/blinkids_v1
Rama principal:	main

Que tiene que hacer el equipo que recibe el proyecto:

- Crear una cuenta/organización en GitHub para el nuevo equipo.
- Pedirle al dueño actual que agregue esa cuenta como colaborador, o que transfiera el repositorio (Settings > Transfer ownership) a la nueva cuenta.
- Alternativa más simple si no se necesita el historial completo: hacer un fork o clonar y subir a un repositorio nuevo propio.

3. Google Cloud Console (inicio de sesión con Google)

IMPORTANTE

Para que sirve en el proyecto:

La app permite iniciar sesión con una cuenta de Google. Las credenciales OAuth (Client ID y Client Secret) que hacen esto posible NO están en el código de la app: están configuradas dentro de Supabase (Authentication > Providers > Google), pero esas credenciales fueron creadas en un proyecto de Google Cloud aparte.

Que tiene que hacer el equipo que recibe el proyecto:

- Pedir acceso al proyecto de Google Cloud donde se crearon esas credenciales (el dueño actual debería saber cuál es), o crear uno nuevo si se prefiere empezar de cero.
- Si se crea uno nuevo: hay que generar nuevas credenciales OAuth 2.0, configurar la pantalla de consentimiento, y agregar el redirect URI que da Supabase, y luego actualizar esas credenciales en Supabase (Authentication > Providers > Google).
- El deep link que usa la app para volver del login (blinkids://callback) ya está configurado en Android e iOS - no hay que tocarlo salvo que cambien el identificador de la app.

4. OneSignal (notificaciones push)

IMPORTANTE

Para que sirve en el proyecto:

Envía las notificaciones push a los celulares (por ejemplo, avisarle al papa que el hijo marco una tarea como hecha). El App ID está embebido en el código de la app (no es secreto, funciona igual que una llave pública), pero la cuenta de OneSignal en sí y la REST API Key (esa sí es secreta) no están en el repo.

Dato actual encontrado en el proyecto:

App ID: 34dd4e8c-d60d-4ee8-8251-c0179f6a87c5

Que tiene que hacer el equipo que recibe el proyecto:

- Pedir que agreguen la cuenta del nuevo equipo como colaborador en el dashboard de OneSignal (onesignal.com) de esa app, o crear una app nueva si prefieren empezar de cero.
- Si crean una app nueva en OneSignal: hay que reemplazar el App ID en el código (lib/data/services/push_notification_service.dart) por el nuevo, y volver a configurar las credenciales de push de Android/iOS dentro de OneSignal.
- De cualquier forma, hay que configurar en Supabase (Edge Functions > send-push > Secrets) las variables ONESIGNAL_APP_ID y ONESIGNAL_REST_API_KEY - sin eso, la función que manda los push falla en silencio.

5. Google Play Console (publicar en Android)

IMPORTANTE

Para que sirva en el proyecto:

Para subir la app a la Play Store (o repartir betas cerradas) hace falta una cuenta de desarrollador de Google Play.

Dato actual encontrado en el proyecto:

Identificador de la app (`com.blinkids.app`)

Que tiene que hacer el equipo que recibe el proyecto:

- Crear una cuenta en play.google.com/console (costo unico de USD 25).
- **IMPORTANTE:** antes de poder publicar hay que generar una llave de firma de release real - ahorita mismo el build de release esta firmado con la llave de debug (ver seccion 2, hallazgo critico). Sin resolver esto no se puede subir un release valido a la Play Store.
- Si ya existe una cuenta de Play Console con esta app publicada o en pruebas, pedir que agreguen al nuevo equipo como usuario en Users and permissions.

6. Apple Developer Program (publicar en iOS)

OPCIONAL

Para que sirva en el proyecto:

Necesario unicamente si se va a distribuir la app en iOS/App Store. Sirve para generar certificados de firma y perfiles de aprovisionamiento.

Dato actual encontrado en el proyecto:

Bundle ID: `com.blinkids.app`

Que tiene que hacer el equipo que recibe el proyecto:

- Crear o pedir acceso a una cuenta en developer.apple.com (costo USD 99/ano).
- Configurar el Bundle ID `com.blinkids.app` en ese equipo de desarrollo.
- Si se usan notificaciones push en iOS, tambien hay que subir el certificado APNs correspondiente dentro del dashboard de OneSignal.
- Este punto se puede posponer si la entrega del miercoles es solo para Android.

2. Revisión de preparación para la migración

Se revisó el estado del repositorio, la configuración de firma de Android, y las alertas de seguridad de Supabase. Esto es lo que se encontró, de más a menos urgente.

CRÍTICO

Hay cambios locales que todavía no se subieron a GitHub

El repositorio local tiene 98 archivos con cambios (81 modificados, 16 nuevos, 1 eliminado) que nunca se hicieron commit. Esto incluye TODO el trabajo de unificación visual, el sistema de misiones de papa con confirmación, los popups de la app, y varios arreglos de esta sesión. Si se entrega el repositorio tal como está en GitHub ahorita, la persona que lo reciba NO va a tener nada de esto - solo va a tener el último commit ("Backup de Supabase..."). Hace falta revisar los cambios, hacer commit y hacer push a origin/main antes de la entrega del miércoles.

CRÍTICO

El build de Android release está firmado con la llave de debug

En `android/app/build.gradle.kts`, la sección `buildTypes > release` usa `signingConfig = signingConfigs.getByName("debug")`. Esto sirve para probar rápido durante el desarrollo, pero un release firmado así NO se puede subir a Google Play, y si alguna vez se reinstala sobre una versión firmada distinta, Android bloquea la actualización. Antes de publicar hay que generar un keystore de verdad (`keytool -genkey`), guardarlo en un lugar seguro (fuera del repositorio), y configurar `signingConfigs > release` con esos datos.

ADVERTENCIA

Protección contra contraseñas filtradas está desactivada

Supabase Auth tiene una opción para rechazar contraseñas que aparecen en bases de datos de filtraciones conocidas (HavelBeenPwned) y está apagada. Se activa en Authentication > Políticas del dashboard de Supabase, sin tocar código. No es bloqueante para la entrega, pero es recomendable activarla antes de lanzar a producción con usuarios reales.

OK

No se encontraron llaves secretas expuestas en el código

La anon key de Supabase y el App ID de OneSignal SI están escritos en el código de la app, pero eso es correcto y esperado: ambos están diseñados para viajar dentro de la app (la seguridad real la da Row Level Security en Supabase, no ocultar esos valores). Las llaves que si son secretas (Google OAuth secret, OneSignal REST API Key) están guardadas del lado del servidor (Supabase), no en el repositorio.

OK

El diseño de seguridad de la base de datos está bien planteado

Se revisaron las alertas automáticas de seguridad de Supabase: no hay ninguna de nivel error, solo advertencias esperables para este tipo de arquitectura (todas las escrituras sensibles pasan por funciones RPC con validaciones propias, en vez de dejar que el cliente escriba directo a las tablas). No se necesita acción antes de la entrega.

OK

No se encontraron otras cuentas o servicios de terceros ocultos

Se revisó el `pubspec.yaml`, la configuración de Android/iOS, y el código en busca de Firebase, servicios de pago, analytics externos (Mixpanel, Amplitude) u otros servicios - no se encontró ninguno. El sistema de analytics del panel de administración está construido enteramente con funciones propias dentro de Supabase, sin depender de un servicio externo.

Checklist para antes del miércoles

- Revisar y hacer commit + push de los cambios locales pendientes.
- Confirmar con el dueño actual como se va a transferir el acceso a Supabase y GitHub (agregar colaborador vs. transferir cuenta).
- Anotar en algún lugar seguro (no en el repo) las credenciales de OneSignal y Google Cloud que use el nuevo equipo.

- Si la entrega incluye un APK para instalar directo (no via Play Store), no es bloqueante seguir firmando con la llave de debug - pero si se va a publicar en Play Store, hay que resolver el keystore de release antes.